

Git Training Manual for Mid-Level Engineers

By Mandeep

Version: 1.0 + 0.2

Duration: 4–6 Hours

Audience: Mid-Level Software Engineers, DevOps Engineers, Technical Leads

Table of Contents

1. Introduction to Git
 2. Repository Management
 3. Daily File Operations
 4. Commit Best Practices
 5. Branching Strategy
 6. Merging and Rebasing
 7. Working with Remote Repositories
 8. Undoing Changes
 9. Stashing and Conflict Resolution
 10. Daily Workflow and Best Practices
-

Page 1 – Introduction to Git

Learning Objectives

By the end of this training, participants will be able to:

- Understand Git fundamentals
- Work effectively with branches
- Collaborate using remote repositories

- Resolve conflicts confidently
- Follow industry-standard Git workflows

What is Git?

Git is a distributed version control system that tracks changes in source code and enables multiple developers to collaborate efficiently.

Benefits

- Source code versioning
- Team collaboration
- Code history tracking
- Easy rollback capability
- Branch-based development

Git Architecture

Working Directory → Staging Area → Local Repository → Remote Repository

Initial Setup

```
git config --global user.name "Your Name" #example mandeep singh avp
git config --global user.email "your.email@company.com" #example mandeep_singhus@live.com

git config --list
```

Page 2 – Repository Management

Create a Repository

```
git init
```

Clone Existing Repository

HTTPS:

```
git clone https://github.com/company/project.git
```

SSH:

```
git clone git@github.com:company/project.git
```

Check Remote Repositories

```
git remote -v
```

Add Remote

```
git remote add origin <repository-url>
```

Change Remote URL

```
git remote set-url origin <new-url>
```

Verify Current Repository State

```
git status
```

Page 3 – Daily File Operations

View File Changes

```
git status
```

```
git diff
```

Add Files to Staging Area

Single File:

```
git add app.js
```

Multiple Files:

```
git add file1.js file2.js
```

All Files:

```
git add .
```

Remove Files

```
git rm filename.txt
```

Rename Files

```
git mv oldname.txt newname.txt
```

Ignore Files

Create a `.gitignore` file:

```
node_modules/  
dist/  
.env  
*.log
```

Page 4 – Commit Operations

Create a Commit

```
git commit -m "Implemented login functionality"
```

Commit All Modified Files

```
git commit -am "Fixed authentication bug"
```

View Commit History

```
git log
```

Compact Format:

```
git log --oneline
```

Graph View:

```
git log --oneline --graph --all
```

View Commit Details

```
git show <commit-id>
```

Amend Previous Commit

```
git commit --amend -m "Updated commit message"
```

Good Commit Message Examples

```
Add user authentication service  
  
Fix payment calculation bug  
  
Refactor customer validation logic
```

Page 5 – Branch Management

View Existing Branches

```
git branch
```

Create Branch

```
git branch feature/user-profile
```

Switch Branch

```
git switch feature/user-profile
```

or

```
git checkout feature/user-profile
```

Create and Switch

```
git checkout -b feature/user-profile
```

Delete Branch

```
git branch -d feature/user-profile
```

Force Delete:

```
git branch -D feature/user-profile
```

Recommended Branch Naming

```
feature/user-management  
feature/payment-api  
bugfix/login-issue  
hotfix/production-error  
release/v2.0
```

Page 6 – Merging and Rebasing

Merge Branch

```
git checkout main  
git merge feature/user-profile
```

Rebase Branch

```
git checkout feature/user-profile  
git rebase main
```

Continue Rebase

```
git rebase --continue
```

Abort Rebase

```
git rebase --abort
```

Merge vs Rebase

Merge	Rebase
Preserves history	Creates cleaner history
Easier for teams	Better for PR cleanup
Creates merge commits	Rewrites commit history

Recommended Practice

Use rebase before creating Pull Requests.

Page 7 – Working with Remote Repositories

Fetch Latest Changes

```
git fetch
```

Pull Changes

```
git pull
```

Specific Branch:

```
git pull origin main
```

Push Changes

```
git push origin feature/user-profile
```

First Push of Branch

```
git push -u origin feature/user-profile
```


Delete Remote Branch

```
git push origin --delete feature/user-profile
```

View Remote Information

```
git remote show origin
```

Page 8 – Undoing Changes

Unstage File

```
git restore --staged app.js
```

Discard Local Changes

```
git restore app.js
```

Revert Commit

```
git revert <commit-id>
```

Soft Reset

```
git reset --soft HEAD~1
```

Mixed Reset

```
git reset HEAD~1
```

Hard Reset

```
git reset --hard HEAD~1
```

Reset to Specific Commit

```
git reset --hard <commit-id>
```

Warning

Avoid `--hard` reset on shared branches.

Page 9 – Stashing and Conflict Resolution

Save Temporary Work

```
git stash
```

View Stashes

```
git stash list
```

Apply Stash

```
git stash apply
```

Apply and Remove

```
git stash pop
```

Delete Stash

```
git stash drop
```

Merge Conflict Example

```
<<<<<< HEAD
Current branch code
=====
Incoming branch code
>>>>>> feature-branch
```

Resolve Conflict

1. Edit file manually
2. Remove conflict markers
3. Save file

```
git add .
git commit -m "Resolved merge conflict"
```

Page 10 – Daily Developer Workflow

Start Your Day

```
git checkout main
git pull origin main
```

Create Feature Branch

```
git checkout -b feature/new-feature
```

During Development

```
git status

git add .

git commit -m "Implemented feature"

git push origin feature/new-feature
```

Sync with Main Branch

```
git fetch origin

git rebase origin/main
```

Before Creating Pull Request

```
git status

git log --oneline

git fetch origin

git rebase origin/main

git push --force-with-lease
```

Most Frequently Used Git Commands

```
git status
git add .
git commit -m "message"
git pull
git push
git fetch
git checkout -b branch-name
```

```
git switch branch-name
git merge branch-name
git rebase main
git stash
git diff
git log --oneline
git revert
git reset
```

Git Best Practices

1. Commit frequently.
2. Write meaningful commit messages.
3. Use feature branches.
4. Pull latest changes before coding.
5. Review code before committing.
6. Keep commits small and focused.
7. Avoid force push on shared branches.
8. Resolve conflicts immediately.
9. Never commit credentials or secrets.
10. Delete merged branches regularly.

Hands-On Lab Exercises

Exercise 1

- Create a repository
- Add a file
- Commit changes

Exercise 2

- Create a feature branch
- Modify code

- Merge into main

Exercise 3

- Create merge conflict
- Resolve conflict manually

Exercise 4

- Stash changes
- Switch branches
- Restore stash

Exercise 5

- Rebase feature branch with main
- Push updated branch

Training Summary

You have learned:

- Git fundamentals
- Branching strategies
- Merge and rebase operations
- Conflict resolution
- Stashing techniques
- Daily Git workflow
- Industry best practices

End of Document